CONF42 PROMPT ENGINEERING 2026

What Running a Front-Facing AI Agent Taught Us About Prompt Engineering

Get the data right. Build the system. Measure every change.

Ahmed Morsi
Product Engineer

Nowlun

Compare. Ship. Track.

Scale hard, with AI agents, not with headcount.

Nowlun

Compare. Ship. Track.

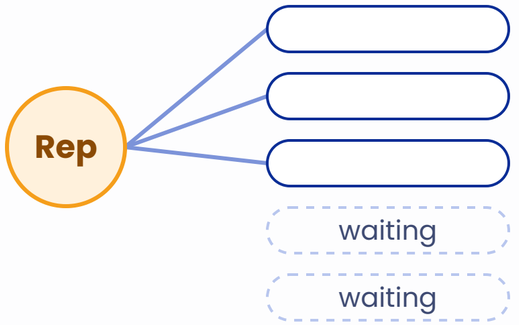
A digital freight platform for sea, air and bulk cargo.

Compare prices across carriers

Ship: book it, handle the papers

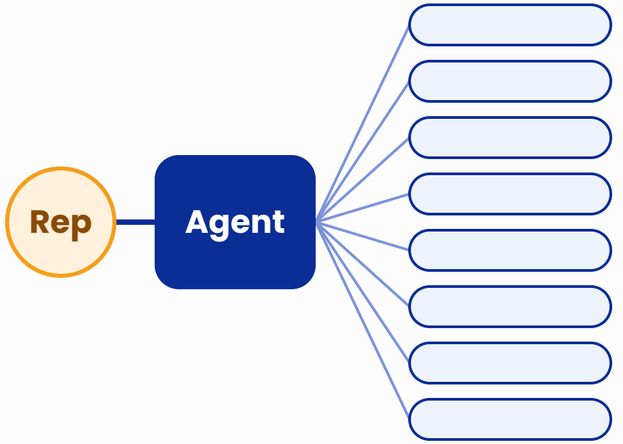
Track every container until it lands

Without agents



One rep, a few chats at a time

With agents



One rep, every chat covered

Each person moves faster

More volume, same team

Same answer, whoever asks

WHAT EACH SIDE BRINGS

A sales rep and an AI agent are good at different things.

The person

Judgement
Reads a messy deal: when to push, when to wait

Trust
Owns the relationship. Customers buy from people.

Context
Knows what was never written down

Accountability
Signs off on the price and the promise

The agent

Speed
Answers in seconds, not minutes

Scale
Every chat, every hour, all at once

Recall
Searches every price and every record

Consistency
The same answer, whoever asks

A rep can’t search every price. An agent shouldn’t own the customer.

OUR COPILOT · A HYBRID SYSTEM

The agent carries the volume. The rep carries the judgement.

Customers
any channel, any hou

Price to Jeddah? · Book it Monday · Where's my box? · Papers attached · Better rate?

The copilot
carries the volume

Understands the ask → **Searches prices** and records → **Drafts the reply,** fills the CRM → **Gate** rules held in code

step-back + reason · *suggestion*

The rep
carries the judgement

Sales rep
approves, edits, or picks up the phone · takes over when the agent steps back

The reply goes out.

A person decided.

Underneath it all: data it can trust · a system that holds the rules · every change measured

THE PRICE IT COULDN’T FIND

The price was in our system. The assistant couldn’t see it.

A sales rep asks

Do we have a price for a 40ft, Shanghai to Jebel Ali?

Assistant

No price on file for this route. Shall I open a request to the pricing team?

35 minutes later, pricing re-quoted a rate we already had.

What was in the system

Saved as	40ft	Status
Shanghai → “Jabal Ali”	$1,850	valid
SHA → “JEA”	$1,910	valid
Shanghai → “Dubai (J. Ali)”	$1,790	valid

The search ran. It looked for “Jebel Ali”, and the prices were saved under three other names.

In one month: 212 pricing requests for routes we already had a price for.

Every answer stands on two layers. The prompt only touches the top one.

Context	What you hand the model this turn: the prompt, the tools, the facts it needs.	**2 · A dictionary it can read**
Data	The records themselves: prices, companies, shipments, requests.	**1 · One thing, one ID**

> “Good context engineering means finding the smallest possible set of high-signal tokens.” Anthropic, Sept 2025

There’s no signal on top without signal underneath. We fixed the layers bottom up.

Map the spellings once, in the data. Never teach them in the prompt.



The same move for everything it looks up

Thing	Its one ID
Port	Standard port code
Company	One record, matched on tax number and email domain
Commodity	Customs code, plus one of 17 labels
Carrier	One carrier code

Searches that missed a price we had: 31% → 4%

Field descriptions are prompts. Write them for a stranger.

Field	What we had	What the assistant reads now
stage = comparing	“comparing”	Collecting quotes, not ready to book. Offer to hold the price; don’t push a booking.
valid_until	“validity”	Last day this price can be quoted. After it, the price is hidden, never shown as old.
incoterm = FOB	“FOB”	Seller delivers to the origin port. Quote from that port; origin trucking is not ours.

One sentence, three readers

The same description feeds the tool description, the hint beside the value, and a new joiner’s onboarding.

Tool calls with a wrong argument

18% → 5%

WHAT THE DATA WORK BOUGHT US

Before we changed a single prompt line, the team got hours back.

212 → 31

pricing requests a month for prices we already had

11 min → 40 s

for a rep to get the right price for a route

×3

requests handled by the same pricing team, no new hires

64% → 93%

of identical questions now get the same answer, whoever asks

A RULE THE PROMPT COULDN’T HOLD

The system mattered more than the prompt.

The business rule Don’t ask the pricing team for a price we already have. Search our prices first; ask only if nothing fits.

A sales rep asks

What’s our price for a 40ft, Shanghai to Jebel Ali?

Assistant

I’ll open a special rate request with the pricing team.

No search ran. Our system already had prices for this route.

In the prompt

```
Search system prices BEFORE opening a special rate request.
```

In capitals, in two sections. It still skipped the search.

In the system

```
REFUSED: not sent. System prices were not searched yet for this request. Search them first.
```

Now it searches first, every time.

PART 2 · THE PLAYBOOK

Build a strong foundation

1

Build the system before the assistant

2

Decide who enforces each rule

3

Give every instruction one home

STEP 1 · SYSTEM BEFORE ASSISTANT

An assistant can only do what the system lets it do.

The job	Before: done by a person, by eye	After: a system rule the assistant can use
Verifying a company	A rep opened the papers and checked them by eye.	Documents in, data extracted, checked against fixed rules. Reps use it too: ×10 verified a month.
Special price requests	A chat message to the pricing team. The price came back in the chat.	A request with a pricing owner, who reads, comments and replies.
Trucking prices	A comment on the sea-freight request. Nobody owned it.	Its own request, sent to the trucking desk for that area.

Before you prompt around a failure, ask: who does this job today, and where?

STEP 2 · WHO ENFORCES WHAT

Code or prompt isn’t a choice. It’s a dial, and every rule gets a setting.

← Code forces it

The model judges it →

Hide the tool

Not offered until the stage allows it

Gate the call

One call off its list, the whole step is refused

Sequence

No price request before a price search

Refuse, then check

Refused up to twice; then a guardrail model re-checks the reason

Hint

“This price is from a nearby port”

Prompt

Pick the price that suits this shipment

STEP 2 · GUARD THE WRITES

A made-up sentence is one bad suggestion. A made-up write spreads.

One sentence nobody wrote

The assistant reads “we ship coal” into a thread where nobody wrote it → Commodity saved: coal → Flagged as a prohibited commodity → Lead stage set to lost → No follow-up; no rep ever calls

What guards the writes now

An email is saved only if it appears in the thread. Stopped 7 of 7 invented contacts.

17 write tools must carry a source quote, fuzzy-matched against the thread.

A company’s classification is picked from 17 fixed labels, never free text.

STEP 2 · THE WAY OUT

Give the assistant a way to step back, and make it say why.

The rep's task list

The assistant stepped back

NO_PRICE_FITS

Importer, ready next week. Our prices don't cover the port. Pricing request drafted for you.

`NEEDS_A_CALL` `SENSITIVE` `NO_TOOL_FOR_THIS` `PAPERS_UNCLEAR`

Job 1 · A safe fallback

The rep gets the reason and everything already known. Nobody starts over.

Job 2 · A signal you can count

The reason comes from a fixed list, so every step-back is a row. Counted weekly, they show where the gaps are.

STEP 3 · ONE HOME PER INSTRUCTION

Your prompt isn’t one prompt. Ours was nine places that never saw each other.

9 places 70 rules 24 conflicts

`system prompt · turn preamble · tool descriptions · tool results · refusals · the rewriter · follow-up seeds · task briefs · context lines`

“IMMEDIATELY disqualify” vs “learn gradually”: it qualified a company on the word “export”.

A trigger asked for a price range. The prompt forbade it; no tool returns one.

A tool result said “tell the rep it’s escalated”. Nothing had escalated it.

A tool showed 5 of 22 allowed values. The model searched the web for the rest.

STEP 3 · WHERE DOES THIS RULE GO?

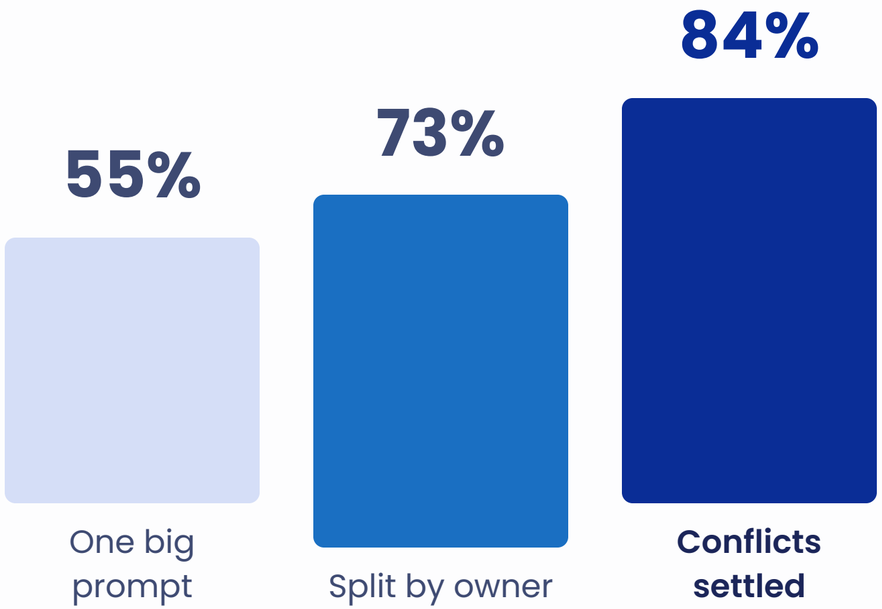
Ask in order. Stop at the first yes.

Code decides | System writes it | Prompt

	The question	Its home	Our example
1	Can code decide it from the data?	A gate or a system rule	`No request before a search`
2	Is it a state field it must read right?	State field + hint	`comparing → not ready`
3	Is it a list of allowed values?	A fixed list in code	`17 commodity labels`
4	Is it about when to use one tool?	Tool description	`For a firm price only`
5	Is it about what a call just returned?	Tool result hint	`Nearby port: say so`
6	Is it a stage's goal, or how its tools work together?	Skill (tone is one too)	`Search, then request`
7	Is it for a special trigger?	Trigger brief	`New price: flag it`
8	Is it a fact about the company?	Knowledge	`Payment terms`
9	Does it hold on every turn, in every stage?	Base prompt	`Never invent a price`

WHAT THE FOUNDATION BOUGHT US

Same 44 test conversations, same model. Passing went from 55% to 84%.



One test conversation

A scripted company that says it exports, and later that it only imports.

Must happen: the assistant doesn't qualify it on the first word.

Graded pass / fail by code

No new model. No longer prompt: **313 lines**, in parts, each with one owner.

Our test set for qualifying a lead · first run 1 try per case, later runs 3

October: a refactor dropped the expiry filter. Expired prices came back. No test noticed; a pricing analyst did.

From lost to a map

1

Where do I start, and in which layer?

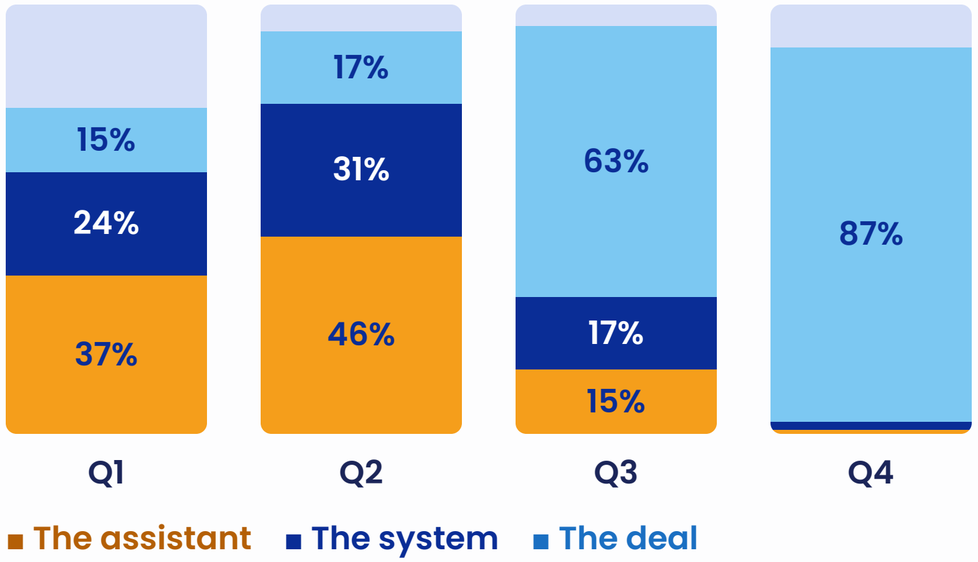
2

Did the fix work? What did it break?

3

What haven’t I seen yet?

Sort every step-back by who it’s about.



A step-back: the assistant stops and hands the task to the rep, with a reason.

About the assistant

Misread the ask, contradicted itself. **Our bug:** fix the prompt or tool, add a test.

About the system

No price for the port, a check that couldn’t run. **Our bug too:** build the piece.

About the deal

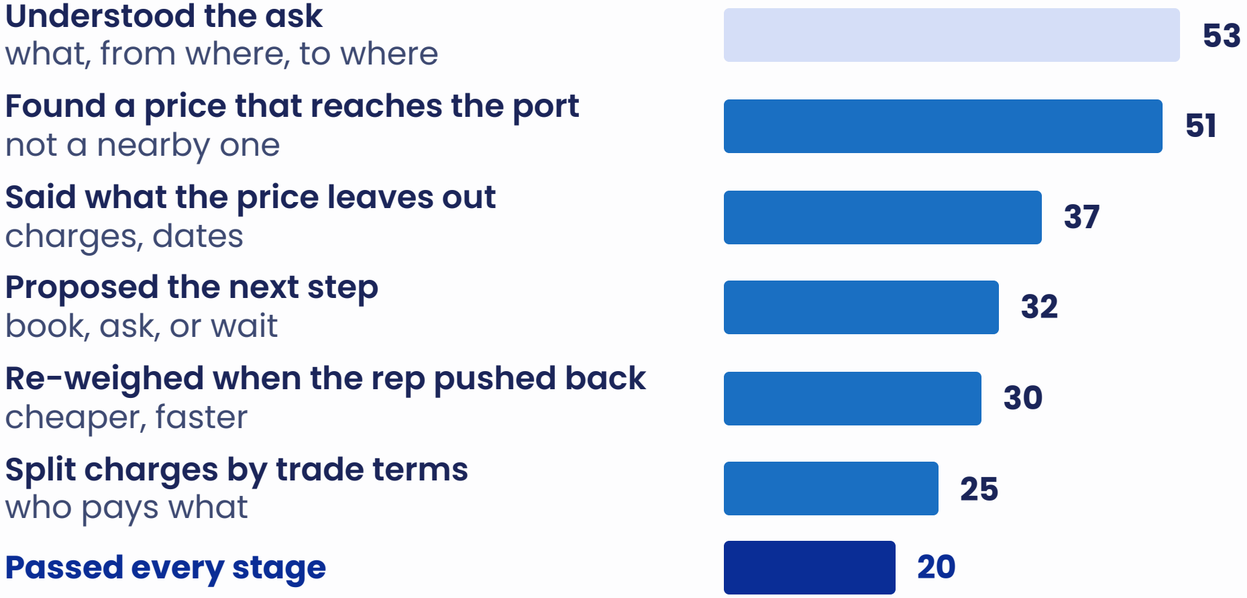
Needs a call, a discount to negotiate. **Real sales work.** Keep it with the rep.

About the assistant: 46% → 1% of step-backs, Q2 to Q4

Q1 · IN WHICH LAYER?

Split the assistant into stages.

53 pricing tests · each stops at the first stage it fails



Idea: “failure as a funnel”, from Bryan Bischof, via Hamel Husain’s evals FAQ

You know where to look

14 tests died reaching the port: a system filter, not a prompt line.

Stages can split again

Until each failure has one obvious owner.

Pick the model per stage

See where a small model is enough. Compare a new model stage by stage.

Q2 · DID THE FIX WORK?

Good test cases start real, get multiplied, and get reviewed.

1

Start from real requests

Pulled from the log. Someone who knows the business reviews each.

→

2

Multiply them

A chat assistant writes variants, good and bad. A person reviews those too.

→

3

Run the evals

Iterate on a cheaper model. Confirm on the live one before you ship.

→

4

Review the judge

Too strict or too lenient? Check it against a person's verdicts.

→

5

Summarise failures

Let a chat assistant group them, with the cases behind each.

6 · Find whose fault it is

The case | The judge | The assistant

Often none of them. Our best findings were

a missing system capability, or a business rule nobody had defined.

Q2 · HOW A TEST IS GRADED

Grade what it did, what it saved, and what it said.

1

What it did

The tools it called, with which arguments, in which order.

`search before request`

2

What it saved

The record as the system holds it at the end: stage, commodities, tags.

`commodities exclude coal`

3

What it said

The message it sent: what must and must not be in it.

`asks which port, briefly`

Code, then a judge

A fake world answers like our backend, so every call has its real effect, and the test reads the end state.

Code first. The judge only grades wording, and counts only once it agrees with a person’s verdicts.

Q2 · TONE IS MEASURED TOO

A suggestion the rep has to rewrite is a wasted suggestion.

Measured against the people it imitates

	Our reps' replies	Suggestions, before
Bubbles per reply	1.6	2.4
Characters, median	~15	238
Bubbles ending in a full stop	1%	79%

Why: three sensible instructions, in three places

The rewriter never saw the conversation. "Keep every fact" meant it could never shorten. "Two messages a turn" counted tool calls, not bubbles.

How we grade it now

Code checks the shape

At most 3 bubbles · at most 2× the customer's message · one question · no lists · no full stop closing every bubble

The judge checks the rest

Says only what this moment needs, and nothing already said

One tone change took 4-bubble replies from 11% to 41%. Nothing measured it. Now this runs on every change.

Q3 · WHAT HAVEN'T I SEEN YET?

Rare failures hide from scripted evals. Replay real requests 1,000 times.

The trap

0 / 20

No failures in 20 runs still allows a true rate near 15%.

Replay the real moment

Temp 1.0: 2 in 999

Temp 1.5, stress: 15 in 1,000

It acted on words nobody wrote: a made-up commodity, a name, a decline over "coal".

The fix, in the tool signature

```
update_company_qualification( commodity, classifications, source_quote, )

source_quote: words copied exactly from the record. Never a summary.

Not in the record → REFUSED: nothing ran. Ask the rep.
```

15 in 1,000 → 0 took effect

THE FINAL SHAPE

Two model boxes out of eleven.

1 Inbox
Messages, follow-ups, notes; one turn at a time

2 Context
Stage, company facts, a summary of older turns

3 Stage picks
Region and stage choose skills and tools

4 Assistant · model
Judgement only: meaning, next step, tool

5 Gate
Allowed at this stage? Source quote real?

6 Tools → records
Every action is a logged row

7 Tone · model
Rewrites the draft; may send it back twice

8 Suggestion
To the rep, links checked; the rep decides

9 People
A hand-off is a task for a person

10 Scheduler
Follow-ups, each with its own brief

11 Measure
Offline, never in the live path

- Code decides
- Model judges
- People

TAKE THIS HOME

The checklist

Get the data right, build the foundation

0 Get the data right: one ID for every thing, and field descriptions written as prompts.

1 Build the system first. Ask who does the job today, and where.

2 Force in code what's costly and checkable; hint what's useful; soften gates you can't be sure of.

3 Give every instruction one home. Tool results are prompts too.

Evolve it

4 Sort step-backs by who they're about. Fix yours first.

5 Split the assistant into stages. Build cases from real requests, multiply them, review them.

6 Replay real requests, hotter than live, to find what you haven't seen.

Thank you

Most of an assistant's reliability lives outside the prompt.

Ahmed Morsi
Product Engineer

Nowlun

Compare. Ship. Track.